Recommender Systems Project Report

on

TopicLens

An Explainable News Recommender

Submitted to Manipal University Jaipur

Towards the partial fulfilment for the Award of the Degree of

B. Tech Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

2026–2027

By

Riddhima
2427010547



MANIPAL UNIVERSITY
JAIPUR

Under the guidance of

Pranshu Pranjal

Department of Artificial Intelligence and Machine Learning

Manipal University Jaipur
Jaipur, Rajasthan

ABSTRACT

News apps decide what people read, but they rarely say why a story was chosen. When the feed gets it wrong, the reader has no way to see the reason or correct it. The objective of this project is to build TopicLens, a news recommender that ranks stories from a reader's click history, chosen topics, reactions to earlier recommendations and direct feedback, and shows the exact reasons behind every pick.

TopicLens is trained on MIND-small, a public dataset of real, anonymised Microsoft News logs with 65,238 articles and 94,057 users. Topics are learned from the article text with non-negative matrix factorisation, and ten features are built for every reader and article pair. Fourteen models were trained and compared on the same time-based split, including KNN, linear and kernel SVMs, a random forest, logistic regression, LightGBM and a neural model trained on a GPU.

On 73,152 test visits the models never saw, a hybrid of LightGBM and the neural model scored best with an AUC of 66.60. Logistic regression reached 63.84, level with LightGBM and the linear SVM, while KNN reached 55.66. The application uses the logistic regression, because its score splits exactly into reasons that can be shown to the reader.

The result is a working web application built with React and TypeScript, with sign-in and storage on Supabase. Every recommendation carries a one-line reason and an optional breakdown, and readers can steer the feed with topic sliders, mutes and "more or less like this". The project shows that a simple, fully explainable model gives up very little accuracy on this task.

Keywords: news recommendation, explainable AI, MIND dataset, logistic regression, topic modelling, Supabase

CONTENTS

ABSTRACT **i**

1 Introduction **1**
1.1 Background and Motivation 1
1.2 Objectives of the Project 1
1.3 Scope of the Work 1
1.4 Product Scenarios 2
1.5 Organisation of the Report 2

2 Requirement Analysis **3**
2.1 Functional Requirements 3
2.2 Non-functional Requirements 3
2.3 Use Case Scenarios 3
2.4 Methodology 4
2.4.1 Dataset 4
2.4.2 Train, validation and test split 4
2.4.3 Models compared 4
2.4.4 Evaluation 5

3 System Design **6**
3.1 Design Goals 6
3.2 System Architecture 6
3.3 Detailed Design 7
3.3.1 Topics and features 7
3.3.2 Ranking and explanation 7
3.3.3 How the four signals are used 7
3.3.4 Database 7

4 Conclusion **9**
4.1 Conclusion 9
4.2 Limitations 9
4.3 Future Work 9

REFERENCES **10**

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Most people now read news through a feed, and the order of that feed is set by a recommender system trained on past clicks. These systems rank well, but they do not explain themselves. A reader cannot tell whether a story is at the top because of their own reading, because it is popular or because it is new. If the feed shows too much of one topic, there is usually no direct way to fix it.

Liu et al. [1] showed that topics learned from article text can be used to explain news recommendations. TopicLens takes that idea and pairs it with a trained click model whose score can be split exactly into its parts, so the explanation shown to the reader is the model's own reasoning and not an approximation of it.

1.2 Objectives of the Project

- Recommend news from four signals: reading and click history, preferred topics, interaction with earlier recommendations and explicit feedback.
- Train real machine learning models on a real dataset, including KNN and SVM, and compare them on one fixed train and test split.
- Pick a model that is accurate and whose score can be explained exactly.
- Show a short reason for every recommendation, with a detailed breakdown on request.
- Let readers sign in and keep their profile, preferences and history across devices.

1.3 Scope of the Work

In scope are: training and evaluating the models offline in Python; a web application with a ranked feed, explanations, topic controls and feedback buttons; a topic explorer; a page that compares the trained ranker with a simple baseline; and accounts with email, Google or guest sign-in.

Out of scope are: live news (the articles are from the MIND dataset, collected in November 2019), a study with real users to measure trust, and running the neural model inside the browser.

1.4 Product Scenarios

Scenario 1: A new reader. Riya signs up with her email address and picks three sections she likes: sports, health and movies. The feed opens with stories from those sections near the top, each marked “a section you chose”.

Scenario 2: Asking why. Riya sees a finance story ranked third and opens “Why am I seeing this?”. The panel shows that most of the score came from the story’s popularity and its similarity to two articles she read earlier, which are listed by name.

Scenario 3: Pushing back. She does not want finance news, so she drags the finance topic slider to zero. The feed reorders at once, and a log shows which stories left her top ten.

1.5 Organisation of the Report

Chapter 2 covers the requirements and the method used to build and evaluate the models. Chapter 3 presents the system design: architecture, ranking model and database. Chapter 4 concludes the report and lists the limitations and future work.

CHAPTER 2: REQUIREMENT ANALYSIS

2.1 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement
FR1	A user can sign in with email and password, Google, or as a guest.
FR2	A new user picks the news sections they like.
FR3	The system shows a ranked feed of news stories for the user.
FR4	Every story in the feed shows a one-line reason and an optional detailed breakdown.
FR5	Opening a story counts as a click and updates the feed.
FR6	The user can mark a story “more like this”, “less like this” or “not interested”.
FR7	The user can raise, lower or mute any topic.
FR8	The user’s profile, preferences and interactions are saved and restored at next sign-in.

2.2 Non-functional Requirements

Table 2.2: Non-functional requirements

Type	Requirement
Accuracy	The ranking model must clearly beat simple baselines on held-out data.
Explainability	The reasons shown must add up exactly to the model’s score.
Speed	The feed must re-rank in under a second after any feedback.
Privacy	A user can only read and change their own data.
Cost	The system must run on free tiers and a single laptop GPU.

2.3 Use Case Scenarios

The two main use cases are getting a recommendation with its reasons, and giving feedback.

Table 2.3: Use case: get and understand a recommendation

Actor	Reader
Precondition	The reader is signed in and has picked sections.
Main flow	1. The system ranks all candidate stories for the reader. 2. The reader opens “Why am I seeing this?” on a story. 3. The system shows each signal’s share of the score and the past reads behind it.
Result	The reader knows why the story was picked.

Table 2.4: Use case: give feedback

Actor	Reader
Precondition	The feed is shown.
Main flow	1. The reader clicks “less like this” or lowers a topic slider. 2. The system updates the scores and re-ranks the feed. 3. The change is saved to the database.
Result	The feed reflects the feedback, now and at the next sign-in.

2.4 Methodology

2.4.1 Dataset

The project uses MIND-small [2], released by Microsoft Research for non-commercial research. It has 65,238 news articles (title, abstract, category and subcategory), 94,057 anonymised users, 156,965 training visits from 9 to 14 November 2019 and 73,152 test visits from 15 November 2019. Each visit lists the stories shown to a user and which ones they clicked. Only 4.05% of shown stories were clicked, so the models are judged on ranking metrics rather than plain accuracy.

2.4.2 Train, validation and test split

The split follows time so that no model sees the future. Models were fitted on visits from 9 to 13 November (126,695 visits), tuned on 14 November (30,270 visits) and tested once on the official test set from 15 November (73,152 visits).

2.4.3 Models compared

Each model scores every shown story, and stories are sorted by score. The models were simple rules (random, popularity, topic match, text match, category match), a KNN classifier ($k = 50$), a linear SVM, a kernel SVM, a random forest, logistic regression, LightGBM

LambdaRank [3], an NRMS-style neural model [4] trained on an RTX 3050 GPU for 4 epochs, and a hybrid of LightGBM with the neural score as an extra feature.

2.4.4 Evaluation

Four standard metrics were used [2]: AUC (how often a clicked story ranks above an unclicked one; 50 is a coin flip), MRR (how high the first click sits) and nDCG@5 and nDCG@10 (how good the top 5 and top 10 are). Table 2.5 shows the main results.

Table 2.5: Results on the 73,152 test visits (higher is better)

Model	AUC	MRR	nDCG@5	nDCG@10
Random	50.07	21.91	22.38	28.66
KNN classifier ($k = 50$)	55.66	25.97	27.85	34.00
Popularity	58.61	27.24	28.66	34.73
Kernel SVM (RBF)	60.40	27.86	30.27	36.82
Random forest	61.79	29.26	31.95	38.15
Neural NRMS (GPU)	63.14	29.75	32.20	38.64
Linear SVM	63.74	30.56	33.52	39.59
LightGBM LambdaRank	63.83	30.58	33.26	39.46
Logistic regression (used in app)	63.84	30.56	33.54	39.61
Hybrid: LightGBM + neural	**66.60**	**32.05**	**35.47**	**41.64**
BATM-NR, from [1]	68.13	32.62	36.36	42.46

Logistic regression, the linear SVM and LightGBM finish within 0.1 AUC of each other, so the extra complexity of trees buys almost nothing here. KNN does poorly because only 4% of rows are clicks, so most neighbours of any row are non-clicks. The hybrid is the most accurate, but it needs the neural encoder at run time and cannot be explained exactly, so the application uses logistic regression.

CHAPTER 3: SYSTEM DESIGN

3.1 Design Goals

- **Recommendation first.** The feed is the main screen; explanations sit behind a single click.
- **Honest explanations.** Every reason shown is a real part of the model's score.
- **Instant feedback.** The model runs in the browser, so the feed re-ranks without a server round trip.
- **Simple to run.** One training script, static files and a managed database.

3.2 System Architecture

Training happens offline in Python. The script exports the articles, the topic model and the model weights as JSON files, which the React application loads. Sign-in and storage go through Supabase (Figure 3.1).

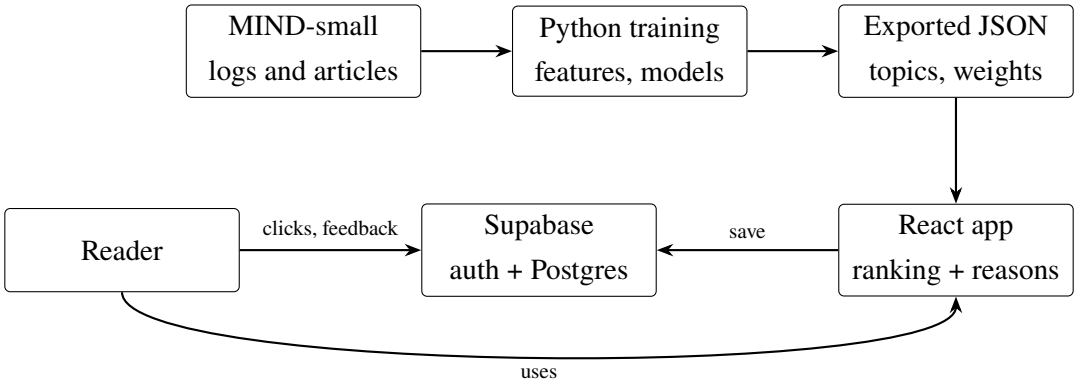


Figure 3.1: System architecture of TopicLens

Table 3.1: Technology stack

Layer	Technology
Machine learning	Python, pandas, scikit-learn [5], LightGBM, PyTorch with CUDA
Front end	React, TypeScript, Vite
Back end	Supabase: PostgreSQL, Auth, row-level security
Version control	Git and GitHub

3.3 Detailed Design

3.3.1 Topics and features

Each article's title and abstract are turned into TF-IDF vectors. Non-negative matrix factorisation finds 30 topics, each described by its top words, and a 100-dimensional LSA space measures how close two articles are in wording. For each reader and story, ten features are computed: topic match, text similarity, category share, subcategory share, earlier-day clicks, earlier-day click rate, times shown before, hours since publication, history length and number of stories shown together.

3.3.2 Ranking and explanation

The application scores each story with the trained logistic regression:

$$s(u, a) = b + \sum_{j=1}^{10} w_j \, z_j(u, a) \tag{3.1}$$

where z_j is the j-th standardised feature for reader u and story a. Each term $w_j z_j$ is shown as that reason's share of the score, so the explanation adds up to the score by construction.

3.3.3 How the four signals are used

1. **Click history:** opening a story adds it to the history, which updates the history-based features straight away.
2. **Preferred sections:** chosen sections add +2.0 to the score of matching stories.
3. **Earlier recommendations:** stories the reader skipped above a click lose 0.5 per skip, up to three skips.
4. **Explicit feedback:** more or less like this, not interested, topic sliders and mutes.

3.3.4 Database

Table 3.2: Main database tables

Table	Contents
profiles	Name, chosen sections and starting history for each user.
preferences	Topic sliders, mutes, liked, disliked and hidden stories, skip counts.
interactions	Append-only log of every click, read, like, dislike, dismiss and skip.

Row-level security lets each user see and change only their own rows, and the interaction log cannot be edited or deleted by users.

CHAPTER 4: CONCLUSION

4.1 Conclusion

TopicLens recommends news from a reader's history, chosen sections, skipped stories and direct feedback, and explains every pick exactly. Of fourteen models tested on the same unseen data, the LightGBM and neural hybrid was the most accurate at 66.60 AUC. Logistic regression scored 63.84, as good as LightGBM and the linear SVM, and was chosen for the application because its reasons are exact.

4.2 Limitations

- The news is from November 2019, so the app is a demonstration, not a live news service.
- The section boost and skip penalty were set by hand, not learned.
- No user study has been run yet, so the effect of explanations on trust is not measured.

4.3 Future Work

- Run a small user study comparing the feed with and without explanations.
- Learn the feedback weights from the logged interactions.
- Bring part of the hybrid's accuracy into the app while keeping explanations exact.

REFERENCES

[1] D. Liu, D. Greene, I. Li, X. Jiang and R. Dong, "Topic-Centric Explanations for News Recommendation", *ACM Transactions on Recommender Systems*, vol. 3, no. 2, 2024, doi:10.1145/3680295.

[2] F. Wu, Y. Qiao, J.-H. Chen, C. Wu, T. Qi, J. Lian, D. Liu, X. Xie, J. Gao, W. Wu and M. Zhou, "MIND: A Large-scale Dataset for News Recommendation", in *Proceedings of the 58th Annual Meeting of the ACL*, 2020, pp. 3597–3606.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye and T.-Y. Liu, "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", in *Advances in Neural Information Processing Systems 30*, 2017.

[4] C. Wu, F. Wu, S. Ge, T. Qi, Y. Huang and X. Xie, "Neural News Recommendation with Multi-Head Self-Attention", in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 6389–6394.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", *Journal of Machine Learning Research*, vol. 12, 2011, pp. 2825–2830.